



Thinking



SproutMentors DPDP & Security Analysis Report

Based on Uploaded Collections and Web App Modules

1. Legal and Compliance Context

SproutMentors processes digital personal data of students, parents, teachers, mentors, educators, and school/institution users. Under India's DPDP framework, the platform should follow principles such as clear notice, consent, purpose limitation, reasonable security safeguards, data retention control, breach response, and rights for data principals. The DPDP Rules, 2025 were notified to operationalize the DPDP Act and include requirements around transparency, consent, data principal rights, breach response, and children's data processing. [Press Information Bureau +1](#)

Because your platform handles children's educational data, parent/guardian consent and role-based access control become very important. The uploaded `students` collection has a unique `admissionNo` index, showing student records are stored as identifiable academic records. [students.metadata.json](#)





2. High-Level Risk Summary

Area	Current Risk	Priority
Student personal + parent data	High	Critical
Student academic performance	High	Critical
Attendance reason like illness	High	Critical
Teacher profile data	Very High	Critical
Teacher bank details	Very High	Critical
Teacher medical information	Very High	Critical
Government ID fields in teacher profile	Very High	Critical
Student login password in sample data	Very High	Critical
Teacher plain password field	Very High	Critical
AI training / AI report data	High	High
Contacts, mentors, educators	Medium/High	High
Class routine and substitutions	Medium	Medium

3. Major Technical Findings



Thinking



3. Major Technical Findings from Collections

3.1 Students Collection

The `students` collection includes:

- Student name
- Roll number
- Admission number
- Class and section
- Gender
- Date of birth
- Date of joining
- Parent names
- Parent mobile numbers
- Parent emails
- Academic performance
- Attendance summary
- Student status
- Login details

The metadata confirms the `students` collection has a unique index on `admissionNo.` `students.metadata.json`

DPDP risk

This collection contains direct personal data of children. It also contains parent/guardian contact



Reply to ChatGPT



3:15

Vo 0.70 5G+ 62



Thinking



Student status

Login details

The metadata confirms the `students` collection has a unique index on `admissionNo`. `students.metadata.json`

DPDP risk

This collection contains direct personal data of children. It also contains parent/guardian contact information. Since most school students are minors, this data should be protected with stricter controls.

Critical issue

The sample student records include `loginDetails.password` in readable/plain format. This should not exist in production.

Required action

Remove plain password from student records.
Store only hashed password using bcrypt or Argon2.
Move authentication data to a separate `users` or `auth_users` collection.
Do not keep password inside the main student profile.
Add parent consent fields.
Add consent timestamp, consent version, consent source, and consent accepted by.

Recommended fields:



Reply to ChatGPT



3:15 PM Add consent fields.

Vo 0.10 5G+ 62
NR KB/s



Thinking

accepted by.



Recommended fields:

JavaScript



```
consent: {  
  parentConsentRequired: true,  
  parentConsentGiven: true,  
  consentGivenBy: "father/mother/guardian",  
  consentDate: Date,  
  consentVersion: "v1.0",  
  privacyPolicyVersion: "v1.0",  
  termsVersion: "v1.0",  
  consentSource: "web-form",  
  consentWithdrawn: false,  
  withdrawnAt: null  
}
```

3.2 Students Attendance Collection

The students_attendances collection includes:

Student ID

Student name

Class

Section

Teacher ID

Date

Attendance status

Reason

Created/updated dates



The metadata confirms an index on studentId +

dat

st



Reply to ChatGPT





Thinking



The metadata confirms an index on `studentId` +
date, which is suitable for attendance lookup by
student/date. `students_attendances.metadata.json`

DPDP risk

Attendance data is personal educational data. The
`reason` field can become sensitive if it stores illness,
family problems, medical reasons, or personal
circumstances.

Required action

Do not allow free-text sensitive details in `reason`.

Instead use controlled categories:

JavaScript



```
reasonCategory: "Sick" | "Family Reason" |  
"Transport Issue" | "Approved Leave" | "Other"  
reasonNote: "optional, restricted visibility"
```

Visibility rule:

Teacher can mark attendance.

Class teacher can view full attendance.

Parent can view own child's attendance.

Student can view own attendance.

Admin/principal can view school-level attendance.

Other teachers should not see unnecessary reason

data



Reply to ChatGPT





Thinking



Visibility rule:

Teacher can mark attendance.

Class teacher can view full attendance.

Parent can view own child's attendance.

Student can view own attendance.

Admin/principal can view school-level attendance.

Other teachers should not see unnecessary reason details.

3.3 Teachers Collection

The `teachers` collection includes:

Email

Mobile

Password

Profile completion status

Created/updated dates

Critical issue

The sample teacher collection includes a `plainPassword` field in at least one record. This is a serious security issue.

Required action



Immediately remove `plainPassword` from all records.



Reply to ChatGPT





Thinking



Required action

Immediately remove plainPassword from all records.

Use:

JavaScript 

```
db.teachers.updateMany({}, { $unset:  
  { plainPassword: "" } })
```

Also ensure password is always hashed.

Recommended:

JavaScript 

```
passwordHash: String  
passwordUpdatedAt: Date  
failedLoginAttempts: Number  
lastLoginAt: Date  
mfaEnabled: Boolean
```

Never store:

plainPassword

passwordText

defaultPassword

temporaryPassword after first login

3.4 Teacher Profiles Collection

+ Reply to ChatGPT



The teacherprofiles collection is the highest-risk
collection in the unloaded files



Thinking



3.4 Teacher Profiles Collection

The `teacherprofiles` collection is the highest-risk collection in the uploaded files.

The metadata confirms the collection has a unique index on `userId`, meaning each teacher profile is linked to one teacher user. `teacherprofiles.metadata.json`

The BSON sample includes:

Teacher personal information

Aadhaar card field

PAN card field

Passport field

Driving license field

Religion

Caste

Category

Date of birth

Marital status

Birthplace

WhatsApp number

Family information

Permanent address

Medical information

Blood group

Chronic disease field

Bank account number

IFSC

MICR

Ac

Te

Admin experience

Personal information



Reply to ChatGPT



Share a link to chat?

This creates a copy that others can chat with



MICR

Academic records

Teaching experience

Admin experience

Research/publication details

Professional memberships

Signature/declaration

DPDP risk

This includes highly sensitive personal information. Even though DPDP uses the term “personal data” broadly rather than old-style sensitive personal data categories, these fields require strict controls because exposure could cause financial, identity, social, employment, or health-related harm.

Critical action

Split teacher profile into separate sections/collections:

JavaScript



```
teacher_basic_profiles  
teacher_professional_profiles  
teacher_bank_details  
teacher_medical_details  
teacher_identity_documents  
teacher_research_projects
```



+ Reply to ChatGPT





Thinking



Do not keep everything in one large document accessible to all admin users.

Access rule

Teacher basic profile: visible to admin/principal/authorized users

Bank details: HR/payroll only

Medical details: HR/authorized medical admin only

Government IDs: HR/compliance only

Research/publication: visible as public/professional profile if teacher consents

Religion/caste/category: collect only if legally or institutionally required; otherwise remove

Encryption

Encrypt at field level:

Bank account number

PAN

Aadhaar

Passport

Driving license

Medical details

Permanent address

Family phone number

Signature



3.5



Reply to ChatGPT



The classes collection includes:



Thinking



3.5 Classes Collection

The `classes` collection includes:

Class ID
Class name
Section
Teacher ID
Monitor details
Location
Schedule
Status
Total students

The metadata confirms `classId` has a unique index.

`classes.metadata.json`

DPDP risk

Class data itself is moderate risk, but the `monitor` object includes a student name, email/image/id. That creates a privacy issue if class data is widely visible.

Required action

Avoid embedding student personal details inside the class document.

Instead use:

JavaScript



Reply to ChatGPT





Thinking



Instead use:

JavaScript



```
monitorStudentId: ObjectId
```

When displaying monitor details, fetch only allowed fields based on the viewer's role.

Avoid storing:

Student email

Student image path

Student admission number

Parent details

inside classes.

3.6 Substitutions Collection

The substitutions collection contains:

Class ID

Date

Time

Substitute teacher name

Reason

Status



Reply to ChatGPT





Thinking



The metadata confirms the collection exists with only the default `_id` index. `substitutions.metadata.json`

DPDP risk

Substitution data may expose teacher absence reasons, staffing issues, or operational school information.

Required action

Do not store detailed personal absence reasons.

Use:

JavaScript



```
reasonCategory: "Teacher Leave" | "Emergency"
| "Admin Adjustment" | "Exam Duty" |
"Training"
internalNote: String // restricted to admin
```

Also add indexes:

JavaScript



```
{ classId: 1, date: 1 }
{ substituteTeacherId: 1, date: 1 }
{ status: 1 }
```



3



Reply to ChatGPT



The metadata confirms Syllabus Progress has an



Thinking



3.7 Syllabus Progress Collection

The metadata confirms `Syllabus_Progress` has an index on `teacherId + classId + subject`, which is suitable for tracking syllabus progress by teacher/class/subject. `Syllabus_Progress.metadata.json`

DPDP risk

Usually low to medium, but it becomes personal data when linked to teacher performance or student learning progress.

Required action

Separate:

Syllabus completion data
Teacher performance analytics
Student performance analytics

Do not show teacher performance dashboards to unauthorized staff.

3.8 Educators, Mentors, Contacts

The uploaded metadata s that educators and mentors both have unique email indexes.



Reply to ChatGPT



The `contacts` collection currently shows only a



Thinking



3.8 Educators, Mentors, Contacts

The uploaded metadata shows that `educators` and `mentors` both have unique email indexes.

`educators.metadata.json``mentors.metadata.json`

The `contacts` collection currently shows only a default `_id` index. `contacts.metadata.json`

DPDP risk

These collections likely store personal contact data, enquiry data, mentor profile data, or professional records.

Required action

For `contacts/enquiries`, add:

Consent checkbox

Purpose notice

Lead source

Privacy policy accepted

Data retention period

Delete request status

Communication opt-in / opt-out

Recommended fields:



JavaScript



Reply to ChatGPT





Thinking



Recommended fields:

JavaScript



```
privacyConsent: {  
  accepted: Boolean,  
  acceptedAt: Date,  
  purpose: "demo-request/contact/newsletter",  
  source: "website-form",  
  privacyPolicyVersion: String  
}
```

3.9 AI Trainings and Insights

The metadata confirms `aitrainings` exists with `default_id` index, while `insights` includes indexes for `slug`, `category type`, `status`, `featured`, and `published date`.

`aitrainings.metadata.json``insights.metadata.json`

DPDP risk

If AI trainings store teacher/student data, prompts, responses, learning records, AI-generated reports, or training participation, then this becomes high-risk.

Required action

Add AI-specific governance:

Do not send unnecessary student identifiers to AI APIs.

Use + Reply to ChatGPT





Thinking



Do not send unnecessary student identifiers to AI APIs.

Use anonymized student IDs where possible.

Do not train external AI models using student data unless consent is clear.

Show AI usage notice before generating reports.

Log AI report generation.

Allow correction of AI-generated reports.

Require teacher/mentor review before final sharing.

4. Answers to the 5 Core Questions

4.1 What data do we collect?

Based on uploaded collections and platform modules, SproutMentors collects or may collect:

Student name, roll number, admission number, class, section, gender, DOB, joining date

Parent/guardian names, mobile numbers, emails

Academic performance, grades, CGPA, percentage, exam records

Attendance records and absence reasons

Student login information

Class schedule, class location, monitor details

Teacher email, mobile, profile completion status

Teacher personal details, family details, address, WhatsApp



Reply to ChatGPT





Thinking



WnatsApp

Teacher professional, academic, research, publication,
patent, award records

Teacher medical details such as blood group, height,
weight, chronic disease field

Teacher bank details

Teacher government ID fields such as Aadhaar, PAN,
passport, driving license

Mentor and educator contact/profile data

Contact form and demo enquiry data

AI training/insight-related content

Syllabus progress

Substitution and routine data

4.2 Why do we collect it?

Data should be collected only for these purposes:

Student academic management

Class and section management

Attendance tracking

Exam, assessment, and report generation

Homework, assignment, and activity management

Parent-school communication

Teacher profile and HR administration

Teacher academic/professional record management

Mentor allocation and counselling

AI-assisted educational insights

Syllabus progress tracking

School administration and compliance

De



Reply to ChatGPT



Each form should mention its specific purpose before



Thinking



School administration and compliance

Demo/contact/newsletter communication

Each form should mention its specific purpose before submission.

4.3 Where is it stored?

Based on the uploaded data format, the system is using MongoDB collections. The `prelude.json` indicates MongoDB Server Version 6.0.27 and tool version 100.14.0. `prelude.json`

Recommended storage classification:

Data Type	Storage
Student profile	MongoDB students collection
Attendance	MongoDB students_attendances collection
Classes/routine	MongoDB classes collection
Teacher login	MongoDB teachers/authentication collection
Teacher full profile	MongoDB teacherprofiles



Reply to ChatGPT



Share a link to chat?

This creates a copy that others can chat with



Attendance	MongoDB students_attendances collection
Classes/routine	MongoDB classes collection
Teacher login	MongoDB teachers/auth collection
Teacher full profile	MongoDB teacherprofiles collection
Documents/photos	Private object storage, not public bucket
Logs	Separate audit_logs collection
AI prompts/reports	Separate ai_logs / ai_reports collection
Backups	Encrypted backup storage

You should document whether data is stored in India, outside India, or processed through third-party services.

4



Reply to ChatGPT





Thinking



4.4 Who can access it?

Recommended access matrix:

Role	Access
Super Admin	Platform configuration only; avoid unnecessary personal data access
School Admin / Principal	Own school data only
Class Teacher	Assigned class students, attendance, homework, reports
Subject Teacher	Assigned subject/class academic data only
Mentor/Counsellor	Assigned students only
Parent	Own child's data only
Student	Own data only
HR/Admin	Teacher HR profile only
Payroll/Finance	Teacher bank details only
Technical Support	Limited support access with approval and logging
Developer	No production personal



Reply to ChatGPT



Important: every API must enforce access control on



Thinking



Payroll/Finance

Teacher bank details only

Technical Support

Limited support access
with approval and
logging

Developer

No production personal
data by default

Important: every API must enforce access control on
backend. Frontend menu hiding is not enough.

4.5 How long do we keep it?

Recommended retention policy:

Data	Suggested Retention
Student profile	Active period + 3 to 5 years
Parent contact details	Until student leaves + defined archival period
Attendance	3 to 5 years
Homework/assignments	Academic year + 1 year
Exams/assessments/ reports	5 years or school policy



Reply to ChatGPT



Share a link to chat?

This creates a copy that others can chat with



defined archival period

Attendance

3 to 5 years

Homework/assignments

Academic year + 1 year

Exams/assessments/
reports

5 years or school policy

Counselling notes

1 to 3 years, restricted

AI-generated reports

1 to 3 years unless
school requires longer

Teacher basic profile

Employment period + 5
years

Teacher bank details

Employment period +
statutory/finance period

Teacher medical data

Only as long as needed

Contact/demo enquiries

6 to 12 months if not
converted

Audit logs

1 to 3 years

Backups

30 to 180 days,
encrypted



Reply to ChatGPT





Thinking



5. Review of the 9 Form Requirements

5.1 Purpose of data collection clearly written

Required on every form.

Example for student registration:

We collect student and parent information to manage admission, class allocation, attendance, academic reports, parent communication, and school administration.

5.2 Consent checkbox

Required for:

Student registration
Parent registration
Teacher profile
Mentor profile
AI report generation
Document upload
Contact/demo forms
Newsletter forms

5.3 Parent/guardian consent where student is under 18



Reply to ChatGPT





Thinking



5.3 Parent/guardian consent where student is under 18

Mandatory for student-related forms.

Add:

JavaScript



```
parentConsent: {  
  required: true,  
  given: true,  
  guardianName: String,  
  relationship: String,  
  mobile: String,  
  email: String,  
  consentAt: Date,  
  consentIp: String  
}
```

5.4 Privacy Policy link

Add near submit button and checkbox.

Example:

☐ I agree to the Privacy Policy and Terms of Use.

5.5 Terms link

Required for students, parents, teachers, mentors, schools, and admins.



5.6 Data usage explanation



Reply to ChatGPT



Each form should say how data will be used.



Thinking



5.6 Data usage explanation

Each form should say how data will be used.

Example:

This data will be used only for academic, administrative, reporting, communication, and student support purposes.

5.7 Document upload warning

Required for profile photo, certificates, marksheets, ID documents, medical documents, teacher documents.

Example:

Upload only the required document. Do not upload unnecessary sensitive personal information.

5.8 Option to withdraw/delete request

Add this to privacy notice:

You may request correction, withdrawal of consent, or deletion of personal data by contacting privacy@sproutsong.com.

5.9 Clear contact il for privacy requests

Use  Reply to ChatGPT





Thinking



5.9 Clear contact email for privacy requests

Use a dedicated email:

privacy@sproutsong.com

Also show it in:

Privacy Policy

Footer

Student/parent forms

Teacher profile forms

AI report generation notice

Document upload pages

6. Module-Wise DPDP Applicability

Module	DPDP Applicability	Ris
Students	Yes	Crit
Classes	Yes	Me
Assignments	Yes	Hig
Reports	Yes	Crit



Reply to ChatGPT



Keywords:

Thinking

Module	DPDP Applicability	Risk	Required Controls
Students	Yes	Critical	Parent consent, RBAC, encryption, no plain password
Classes	Yes	Medium	Avoid embedded student personal data
Assignments	Yes	High	Student-only visibility, teacher/class access
Reports	Yes	Critical	Download logs, parent/student restriction, AI disclaimer
Exams	Yes	Critical	Marks edit logs, result publish workflow
Assessments	Yes	Critical	Consent, restricted analytics
Activities	Yes	Medium/High	Media consent, student visibility control
Homework	Yes	High	Class-based access, retention
Teacher Profile	Yes	Critical	Split sensitive data, encrypt bank/medical/ID fields
Attendance	Yes	Critical	Controlled reason categories, edit logs
Tasks / Projects	Yes	Medium	Assigned-user access only
Class Routine	Yes	Low/Medium	Teacher/class visibility control
Substitutions	Yes	Medium	Avoid personal absence reasons
Syllabus Progress	Yes	Medium	Teacher performance privacy
Mentors	Yes	High	Assigned-student access, consent
Educators	Yes	Medium/High	Profile consent, email protection
Contacts / Demo Requests	Yes	Medium	Consent, purpose, retention
AI Trainings / AI Reports	Yes	Critical	AI notice, anonymization, human review



Thinking



7. Critical Security Gaps to Fix First

7.1 Remove Plain Passwords

This is the highest-priority fix.

Issues observed:

Student login details contain password field.

Teacher records contain `plainPassword` field in sample data.

Immediate cleanup:

JavaScript



```
db.students.updateMany({}, { $unset:
{ "loginDetails.password": "" } })
db.teachers.updateMany({}, { $unset:
{ plainPassword: "" } })
```

Recommended auth design:

JavaScript



```
{
  userId,
  role,
  email,
  mobile,
```



Reply to ChatGPT



```
lastLoginAt,
passwordUpdateAt
```



Thinking



Recommended auth design:

JavaScript



```
{  
  userId,  
  role,  
  email,  
  mobile,  
  username,  
  passwordHash,  
  mfaEnabled,  
  lastLoginAt,  
  passwordUpdatedAt,  
  status  
}
```

7.2 Separate Authentication from Profile Data

Do not keep login details inside `students`.

Better structure:

JavaScript



```
users  
students  
teachers  
teacherprofiles  
parents  
roles  
permissions
```



Ex



Reply to ChatGPT





Thinking



Example:

JavaScript



```
users {  
  _id,  
  role: "student",  
  linkedProfileId,  
  email,  
  mobile,  
  passwordHash,  
  status  
}
```

7.3 Add School/Tenant Isolation

Most collections should include:

JavaScript



```
schoolId  
academicYearId  
createdBy  
updatedBy
```

Without `schoolId`, multi-school data separation becomes weak.

Add to:



students

classes

st

teachers

teacherprofiles



Reply to ChatGPT





Thinking



Add to:

students

classes

students_attendances

teachers

teacherprofiles

substitutions

Syllabus_Progress

assignments

reports

exams

assessments

homework

tasks

Every query should filter by `schoolId`.

Example:

JavaScript



```
db.students.find({  
  schoolId: req.user.schoolId,  
  class: req.user.assignedClass  
})
```

7.4 Add Audit Log



Cre



Reply to ChatGPT



JavaScript



Thinking



7.4 Add Audit Logs

Create:

JavaScript



```
audit_logs {  
  actorId,  
  actorRole,  
  schoolId,  
  module,  
  action,  
  recordType,  
  recordId,  
  before,  
  after,  
  ipAddress,  
  userAgent,  
  createdAt  
}
```

Log these actions:

View student profile
Edit student data
Delete/archive student
Download report
Generate AI report
View teacher bank details
View teacher medical details
Update marks
Update attendance
Export CSV/PDF
Change role/permission
Login/logout
Forgot password
Password reset



Reply to ChatGPT





Thinking



Export CSV/PDF

Change role/permission

Login/logout

Failed login

Password reset

7.5 Encrypt Sensitive Fields

Encrypt these fields:

Student parent mobile/email

Student DOB

Attendance reason notes

Teacher bank account number

Teacher PAN/Aadhaar/passport/license

Teacher medical info

Teacher family details

Teacher permanent address

Counselling notes

AI reports

Uploaded documents

Use application-level encryption or MongoDB

Client-Side Field Level Encryption if available.

8. Recommended Index Improvements



Reply to ChatGPT





Thinking



8. Recommended Index Improvements

Students

Current: admissionNo unique. `students.metadata.json`

Add:

JavaScript



```
{ schoolId: 1, admissionNo: 1 } unique  
{ schoolId: 1, class: 1, section: 1 }  
{ schoolId: 1, status: 1 }  
{ "parents.fatherMobile": 1 }  
{ "parents.motherMobile": 1 }
```

If multi-school, avoid global unique admission number. Admission number should be unique per school, not globally.

Students Attendance

Current: studentId + date. `students_attendances.metadata.json`

Add:



JavaScript



Reply to ChatGPT





Thinking



Students Attendance

Current: `studentId + date.` `students_attendances.metadata.json`

Add:

JavaScript



```
{ schoolId: 1, class: 1, section: 1, date: 1 }  
{ teacherId: 1, date: 1 }  
{ status: 1, date: 1 }
```

Classes

Current: `classId` unique. `classes.metadata.json`

Better:

JavaScript



```
{ schoolId: 1, classId: 1 } unique  
{ schoolId: 1, teacherId: 1 }  
{ schoolId: 1, status: 1 }
```

Teacher Profiles



Current: `userId` unique. `teacher_profiles.metadata.json`



Reply to ChatGPT



Add



Thinking



Teacher Profiles

Current: userId unique. `teacherprofiles.metadata.json`

Add:

JavaScript



```
{ schoolId: 1, userId: 1 } unique  
{ schoolId: 1, "teacherInfo.email": 1 }  
{ schoolId: 1, status: 1 }
```

Substitutions

Current: only _id. `substitutions.metadata.json`

Add:

JavaScript



```
{ schoolId: 1, classId: 1, date: 1 }  
{ schoolId: 1, substituteTeacherId: 1, date:  
1 }  
{ schoolId: 1, status: 1 }
```

Syllabus Progress



Reply to ChatGPT





Thinking



Syllabus Progress

Current: teacherId + classId + subject.

Syllabus_Progress.metadata.json

Add:

JavaScript



```
{ schoolId: 1, academicYearId: 1, classId: 1,  
  subject: 1 }
```

9. Recommended Schema Enhancements

9.1 Common Fields for All Personal Data Collections

Add to all major collections:

JavaScript



```
schoolId: ObjectId,  
academicYearId: ObjectId,  
createdBy: ObjectId,  
updatedBy: ObjectId,  
deletedAt: Date,  
deletedBy: ObjectId, ↓  
isDeleted: Boolean,  
dataSource: String,
```



Reply to ChatGPT





Thinking

Collections:



JavaScript



```
schoolId: ObjectId,  
academicYearId: ObjectId,  
createdBy: ObjectId,  
updatedBy: ObjectId,  
deletedAt: Date,  
deletedBy: ObjectId,  
isDeleted: Boolean,  
dataSource: String,  
consentVersion: String,  
retentionCategory: String
```

9.2 Consent Collection

Create separate collection:

JavaScript



```
consents {  
  dataPrincipalType: "student" | "parent" |  
  "teacher" | "mentor",  
  dataPrincipalId,  
  guardianId,  
  purpose,  
  consentText,  
  consentVersion,  
  privacyPolicyVersion,  
  termsVersion,  
  accepted: Boolean,  
  acceptedAt,  
  withdrawnAt,  
  ipAddress,  
  userAgent,  
  source  
}
```



Reply to ChatGPT





Thinking



9.3 Data Requests Collection

Create:

JavaScript



```
data_requests {  
  requesterId,  
  requesterType,  
  requestType: "access" | "correction" |  
  "deletion" | "withdrawal" | "grievance",  
  relatedStudentId,  
  description,  
  status: "pending" | "in_review" |  
  "completed" | "rejected",  
  assignedTo,  
  response,  
  createdAt,  
  closedAt  
}
```

9.4 AI Processing Logs

Create:

JavaScript



```
ai_processing_logs {  
  schoolId,  
  studentId,  
  requestedBy,  
  module,  
  purpose,  
  ...  
}
```



Reply to ChatGPT





Thinking



9.4 AI Processing Logs

Create:

JavaScript



```
ai_processing_logs {  
  schoolId,  
  studentId,  
  requestedBy,  
  module,  
  purpose,  
  inputDataFields,  
  anonymized: Boolean,  
  provider,  
  model,  
  outputRecordId,  
  humanReviewed: Boolean,  
  reviewedBy,  
  createdAt  
}
```

Do not store full AI prompts with unnecessary personal data unless required.

10. Privacy Notice Text for Forms

Student Registration

We collect student and parent/guardian information to manage class allocation



Reply to ChatGPT



Responses generated by ChatGPT will be accessed only by authorized school staff.



Thinking



10. Privacy Notice Text for Forms

Student Registration

We collect student and parent/guardian information to manage class allocation, attendance, academic records, communication, reports, and school administration. This data will be accessed only by authorized school staff, teachers, parents/guardians, and platform administrators as required for educational purposes.

Attendance

Attendance data is collected to track student presence, absence, leave, and academic participation. Absence reasons should be limited to necessary information only.

Teacher Profile

We collect teacher profile information for employment, academic, administrative, communication, and institutional record purposes. Sensitive details such as bank, identity, family, and medical information will be restricted to authorized administrative users only.

AI



Reply to ChatGPT



Student academic data may be used to generate



Thinking



AI Report

Student academic data may be used to generate AI-assisted educational insights and reports. AI output is only a support tool and should be reviewed by authorized teachers or mentors before being used for academic decisions.

Document Upload

Upload only documents required for academic or administrative purposes. Do not upload unnecessary sensitive personal information. Uploaded documents will be stored securely and accessed only by authorized users.

11. Final Prioritized Action Plan

Phase 1 — Critical Security Cleanup

Remove all plain password fields.

Move authentication to a separate users/auth collection.

Hash all passwords.

Add MFA for admin/principal/super admin.

Block direct access to student/teacher data without backend role check.



Reply to ChatGPT



3:21



NR 0.00 KB/s 5G+ 60



Thinking



Phase 2 – DPDP Form Compliance

- Add purpose notice to every form.
- Add consent checkbox.
- Add parent/guardian consent for minors.
- Add Privacy Policy and Terms links.
- Add document upload warnings.
- Add privacy contact email.

Phase 3 – Database Restructuring

- Add `schoolId` and `academicYearId`.
- Split teacher sensitive data into separate collections.
- Remove embedded student monitor details from classes.
- Replace free-text sensitive reasons with categories.
- Add soft-delete and retention fields.

Phase 4 – Access Control

Implement role-based and school-based authorization.

Required policy:

Plain text



User can access record only if:

1. user belongs to same school,
2. user role has permission for module,
3. user is assigned to class/student/subject where required.



Reply to ChatGPT





Thinking



Required policy:

Plain text



User can access record only if:

1. user belongs to same school,
2. user role has permission for module,
3. user is assigned to class/student/subject where required,
4. sensitive field access is separately allowed.

Phase 5 – Audit and Monitoring

Create audit logs.

Track view/edit/delete/download/export/generate actions.

Add admin dashboard for privacy/security events.

Alert on bulk export or repeated failed access.

Phase 6 – AI Governance

Add AI consent.

Anonymize student data before AI processing.

Log AI usage.

Require human review.

Allow correction of AI reports.

Do not use student data for AI training unless specifically consented.



12 Final Conclusion



Reply to ChatGPT



Your current system structure is workable, but before



Thinking



12. Final Conclusion

Your current system structure is workable, but before scaling across schools, SproutMentors should strengthen DPDP compliance at the **schema, API, access-control, and consent levels**.

The biggest immediate risks are:

1. Plain password fields in student/teacher records.
2. Highly sensitive teacher profile data stored in one large document.
3. Lack of visible consent/retention fields in the uploaded schemas.
4. Attendance reasons and medical/family/government ID data requiring restricted access.
5. Need for school-level data isolation across all modules.
6. Need for audit logs for every sensitive action.

Once these are fixed, SproutMentors will be much stronger for school onboarding, parent trust, DPDP readiness, and future AI-based education features.



Sources



Reply to ChatGPT





Thinking



Sources

1. If any relevant fields are missing or not?
2. Check the types and suggest the changes considering security point of view
3. Does Captcha code provides the security from the front-end?
4. Direct API call from browser or terminal through curl call can be prevented?
5. Any server side validation to prevent the attack ?
6. Middle ware implementation and security check - with node.js?
7. Mongodb - data protection

Answer these in correspondence to all the files, and give the technical viewpoint, not just conceptual understanding

Thought for 58s >

Yes — now we can go technical. Based on the uploaded BSON samples and metadata, the main issue is not only "DPDP compliance"; it is also



Reply to ChatGPT



middleware, and MongoDB-level protection



Thinking



1. Relevant Fields Missing or Not?

A. Common fields missing across most collections

For DPDP, security, auditing, and multi-school safety, most collections should have these fields:

JavaScript



```
schoolId: ObjectId,  
academicYearId: ObjectId,  
createdBy: ObjectId,  
updatedBy: ObjectId,  
deletedBy: ObjectId,  
deletedAt: Date,  
isDeleted: Boolean,  
status: String,  
dataSource: String,  
ipAddress: String,  
userAgent: String,  
retentionUntil: Date,  
consentRefId: ObjectId,  
privacyPolicyVersion: String,  
termsVersion: String
```

Why this matters

Right now, many records are identifiable but do not clearly show:

Who created it

Who updated it

Wh

Wh

Whether consent exists



Reply to ChatGPT





Thinking



Why this matters

Right now, many records are identifiable but do not clearly show:

Who created it

Who updated it

Which school owns it

Which academic year it belongs to

Whether consent exists

When it should be deleted

Whether it is soft-deleted

Which user/IP performed the action

For a multi-school SaaS like SproutMentors, `schoolId` is critical. Without `schoolId`, every query becomes risky because data isolation depends only on application logic.

B. Students collection – missing fields

Current sample contains useful fields like name, roll no, admission no, class, section, gender, DOB, parents, academic performance, attendance, status, and login details.

Missing or should be added



JavaSeries



Reply to ChatGPT



academicYearId

studentCode



Thinking



no, admission no, class, section, gender, DOB, parents, academic performance, attendance, status, and login details.

Missing or should be added:

JavaScript



```
schoolId
academicYearId
studentCode
parentGuardianConsent
consentVersion
privacyPolicyVersion
dataProcessingPurpose
isMinor
guardianVerified
createdBy
updatedBy
isDeleted
deletedAt
retentionUntil
lastProfileReviewAt
```

Critical issue

`loginDetails.password` should not exist inside student profile. It should be removed and moved to a separate auth collection as a hash only.

Better:

JavaScript



```
users {
  _id,
  linkedProfileId,
```



Reply to ChatGPT





Thinking



Better:

JavaScript



```
users {  
  _id,  
  linkedProfileId,  
  role: "student",  
  username,  
  email,  
  mobile,  
  passwordHash,  
  mfaEnabled,  
  lastLoginAt,  
  passwordUpdatedAt,  
  status  
}
```

C. Teachers collection – missing fields

Current sample has email, mobile, password, plainPassword, profile completion status.

Missing:

JavaScript



```
role  
schoolId  
isEmailVerified  
isMobileVerified  
passwordUpdatedAt  
lastLoginAt  
failedLoginAttempts  
lockedUntil
```



Reply to ChatGPT





Thinking



Missing:

JavaScript



```
role
schoolId
isEmailVerified
isMobileVerified
passwordUpdatedAt
lastLoginAt
failedLoginAttempts
lockedUntil
mfaEnabled
status
createdBy
```

Critical issue

plainPassword exists in uploaded teacher records.
This must be removed immediately.

JavaScript



```
db.teachers.updateMany({}, { $unset:
{ plainPassword: "" } })
```

Also rename password to passwordHash for clarity.

D. Teacher Profiles collection — missing security separation



The teacher profile BSON contains highly sensitive

data
de



Reply to ChatGPT



caste, religion, address, academic records, and



Thinking



D. Teacher Profiles collection – missing security separation

The teacher profile BSON contains highly sensitive data: Aadhaar, PAN, passport, driving license, bank details, medical information, family information, caste, religion, address, academic records, and declaration.

This collection is too broad. It should be split.

Recommended structure:

JavaScript



```
teacher_basic_profiles
teacher_professional_profiles
teacher_identity_documents
teacher_bank_details
teacher_medical_details
teacher_family_details
teacher_research_profiles
```

Why

A principal may need academic/professional info, but not bank account or Aadhaar.

Payroll may need bank details, but not medical history.

HR may need identity documents, but not classroom analytics.

Teachers may update their profile, but should not be able to update classroom analytics.



Reply to ChatGPT





Thinking



not bank account or Aadhaar.

Payroll may need bank details, but not medical history.

HR may need identity documents, but not classroom analytics.

Teachers may update their profile, but should not be able to modify verified identity data after approval.

E. Attendance collection – missing fields

Current attendance sample includes date, studentId, studentName, class, section, status, reason, teacherId.

Missing:

JavaScript



```
schoolId  
academicYearId  
markedBy  
approvedBy  
isEdited  
editHistory  
reasonCategory  
reasonVisibility  
parentNotified  
notificationSentAt
```

Security issue



Reply to ChatGPT





Thinking



Security issue

reason is free text. This can expose medical or family data such as "fever", "family issue", "hospital", etc.

Better:

JavaScript



```
reasonCategory: {  
  type: String,  
  enum: ["SICK", "TRANSPORT", "FAMILY_REASON",  
    "APPROVED_LEAVE", "SCHOOL_EVENT", "OTHER"]  
},  
reasonNote: {  
  type: String,  
  select: false  
}
```

Keep reasonNote hidden by default and expose it only to authorized roles.

F. Classes collection – missing fields

Current class records include classId, className, section, teacherId, monitor, location, schedule, status, totalStudents.

Missing:



JavaScript



Reply to ChatGPT





Thinking



Missing:

JavaScript



```
schoolId  
academicYearId  
classTeacherId  
createdBy  
updatedBy  
effectiveFrom  
effectiveTo
```

Security issue

The `monitor` field embeds student name, email, image, and admission ID inside the class document.

Better:

JavaScript



```
monitorStudentId: ObjectId
```

Fetch monitor details only if the requesting user has permission.

G. Substitutions collection – missing fields



Current substitution records include classId, date,

tin



Reply to ChatGPT



Missing:



Thinking



G. Substitutions collection – missing fields

Current substitution records include classId, date, time, substituteTeacherName, reason, status.

Missing:

JavaScript



```
schoolId  
academicYearId  
originalTeacherId  
substituteTeacherId  
subject  
createdBy  
approvedBy  
approvalStatus  
reasonCategory  
internalNote
```

Avoid storing only substituteTeacherName. Use substituteTeacherId.

H. Syllabus Progress collection – missing fields

Metadata shows index on teacherId, classId, and subject. `Syllabus_Progress.metadata.json`

Add:



Reply to ChatGPT





Thinking



Add:

JavaScript



```
schoolId
academicYearId
termId
subjectId
chapterId
completionPercentage
lastUpdatedBy
verifiedBy
studentImpactFlag
```

This helps separate academic progress from teacher performance analytics.

I. Contacts, mentors, educators

Metadata shows educators and mentors have unique email indexes, while contacts has only default `_id`.

educators.metadata.json

mentors.metadata.json

contacts.metadata.json

For these collections, add:

JavaScript



```
consentAccepted
consentAcceptedAt
purpose
source
communicationOptIn
privacyPolicyAccepted
```



Reply to ChatGPT





Thinking



JavaScript



```
consentAccepted
consentAcceptedAt
purpose
source
communicationOptIn
privacyPolicyVersion
retentionUntil
deleteRequested
deleteRequestedAt
```

For contacts, add useful indexes:

JavaScript



```
{ email: 1 }
{ mobile: 1 }
{ createdAt: -1 }
{ status: 1 }
```

2. Type Check and Suggested Changes from Security Viewpoint

A. Use ObjectId instead of string IDs

Current samples use many string-based references:

JavaScript



Reply to ChatGPT





Thinking



A. Use ObjectId instead of string IDs

Current samples use many string-based references:

JavaScript



```
teacherId: "6948ca6581e2ea0a70e454b4"  
studentId: "695646fc5229413df5260b6c"  
classId: "695646fc5229413df5260b56"
```

Better:

JavaScript



```
teacherId: {  
  type: mongoose.Schema.Types.ObjectId,  
  ref: "Teacher",  
  required: true,  
  index: true  
}
```

Why:

ObjectId validation becomes easier.

MongoDB relations are cleaner.

Query performance improves.

It reduces spoofed string IDs.

Authorization checks become more reliable.

B. Dates should be Date type, not string



Reply to ChatGPT



Fields like dob, dateOfJoin, year, from, to, and date should be consistently typed

3:23

Authorization checks become more reliable.

NR 0.00 KB/s 5G+ 60



Thinking



B. Dates should be Date type, not string

Fields like `dob`, `dateOfJoin`, `year`, `from`, `to`, and `date` should be consistently typed.

Better:

JavaScript



```
dob: Date,  
dateOfJoining: Date,  
attendanceDate: Date,  
createdAt: Date,  
updatedAt: Date
```

For academic year only, use string enum:

JavaScript



```
academicYear: "2025-2026"
```

For publication year:

JavaScript



```
year: Number
```



Reply to ChatGPT





Thinking



C. Mobile numbers should be strings, not numbers

Keep mobile as string:

JavaScript



```
mobile: {  
  type: String,  
  match: /^\\+?[1-9]\\d{7,14}$/  
}
```

Why:

Mobile numbers can start with 0.

Country code is needed.

Numeric type loses formatting.

Phone number is not used for arithmetic.

D. Bank account, Aadhaar, PAN, passport must be strings and encrypted

Never store these as plain text.

Bad:

JavaScript



Reply to ChatGPT





Thinking



D. Bank account, Aadhaar, PAN, passport must be strings and encrypted

Never store these as plain text.

Bad:

JavaScript



```
accountNumber: "66018356550"  
aadhaarCard: "248238402055"  
panCard: "BNOPD4509B"
```

Better:

JavaScript



```
bankDetails: {  
  accountNumberEnc: String,  
  accountNumberLast4: String,  
  ifsc: String  
}  
  
identity: {  
  aadhaarEnc: String,  
  aadhaarLast4: String,  
  panEnc: String,  
  panMasked: String  
}
```

Application should show only masked values:

Plain text



Reply to ChatGPT



Account: XXXXXXXX6550



Thinking



JavaScript



```
accountNumber: "66018356550"  
aadhaarCard: "248238402055"  
panCard: "BNOPD4509B"
```

Better:

JavaScript



```
bankDetails: {  
  accountNumberEnc: String,  
  accountNumberLast4: String,  
  ifsc: String  
}  
  
identity: {  
  aadhaarEnc: String,  
  aadhaarLast4: String,  
  panEnc: String,  
  panMasked: String  
}
```

Application should show only masked values:

Plain text



```
Aadhaar: XXXX-XXXX-2055  
PAN: BNOPD****B  
Account: XXXXXXXX6550
```

E. Password field



Current:



Reply to ChatGPT



JavaScript





Thinking



E. Password fields

Current:

JavaScript



```
password
plainPassword
loginDetails.password
```

Better:

JavaScript



```
passwordHash: {
  type: String,
  select: false,
  required: true
}
```

Remove forever:

JavaScript



```
plainPassword
loginDetails.password
defaultPassword
tempPassword
```

Temporary passwords should be sent once and never stored as readable text.



Reply to ChatGPT



F. Status fields should be enum



Thinking



F. Status fields should be enum

Instead of free string:

JavaScript



```
status: "Active"
```

Use:

JavaScript



```
status: {  
  type: String,  
  enum: ["ACTIVE", "INACTIVE", "PENDING",  
    "SUSPENDED", "ARCHIVED"],  
  default: "ACTIVE"  
}
```

Attendance status:

JavaScript



```
status: {  
  type: String,  
  enum: ["PRESENT", "ABSENT", "LATE",  
    "HALF_DAY", "LEAVE"],  
  required: true  
}
```

Substitution status:



Reply to ChatGPT





Thinking



Substitution status:

JavaScript



```
status: {  
  type: String,  
  enum: ["PENDING", "APPROVED", "REJECTED",  
    "COMPLETED", "CANCELLED"]  
}
```

G. Academic performance type changes

Current:

JavaScript



```
cgpa: 8.5,  
percentage: 85,  
grade: "A"
```

Better:

JavaScript



```
academicPerformance: [{  
  examId: ObjectId,  
  subjectId: ObjectId,  
  marksObtained: Number,  
  totalMarks: Number,  
  percentage: Number,  
}]
```



Reply to ChatGPT





Thinking



Better:

JavaScript



```
academicPerformance: [{
  examId: ObjectId,
  subjectId: ObjectId,
  marksObtained: Number,
  totalMarks: Number,
  percentage: Number,
  grade: {
    type: String,
    enum: ["A+", "A", "B+", "B", "C", "D",
"F"]
  },
  published: Boolean,
  publishedAt: Date
}]
```

Do not store sensitive report comments in the same object unless access-controlled.

H. Teacher profile sensitive fields should be `select: false`

Example:

JavaScript



```
bankDetails: {
  accountNumberEnc: { type: String, select:
false },
  ifsc: { type: String, select: false }
},
medicalInfo: {
```



Reply to ChatGPT





Thinking



H. Teacher profile sensitive fields should be `select: false`

Example:

JavaScript

```
bankDetails: {  
  accountNumberEnc: { type: String, select:  
false },  
  ifsc: { type: String, select: false }  
},  
medicalInfo: {  
  chronicDiseasesEnc: { type: String, select:  
false }  
}
```

This prevents accidental exposure from normal API responses.

3. Does CAPTCHA Code Provide Security from Frontend?

No. CAPTCHA on the frontend alone does not provide real security.

Frontend CAPTCHA only es that the user completed a visual/token challenge in the browser. But an attacker can skip your frontend completely and dir



Reply to ChatGPT





Thinking



3. Does CAPTCHA Code Provide Security from Frontend?

No. CAPTCHA on the frontend alone does not provide real security.

Frontend CAPTCHA only proves that the user completed a visual/token challenge in the browser. But an attacker can skip your frontend completely and directly hit your API using:

Bash



```
curl  
Postman  
Python script  
Browser dev tools  
Bot script
```

Correct CAPTCHA implementation

CAPTCHA must be verified on the backend.

Flow:

1. Frontend receives CAPTCHA token.
2. Frontend sends token with form data.
3. Backend sends token to CAPTCHA provider

verification API



Reply to ChatGPT





Thinking



1. Frontend receives CAPTCHA token.
2. Frontend sends token with form data.
3. Backend sends token to CAPTCHA provider verification API.
4. Backend checks success score/result.
5. Backend accepts or rejects request.

Example backend middleware:

JavaScript



```
const axios = require("axios");

async function verifyCaptcha(req, res, next) {
  try {
    const token = req.body.captchaToken;

    if (!token) {
      return res.status(400).json({
        success: false,
        message: "Captcha token is required"
      });
    }

    const response = await axios.post(
      "https://challenges.cloudflare.com/turnstile/v0/siteverify",
      new URLSearchParams({
        secret:
          process.env.TURNSTILE_SECRET_KEY,
        response: token,
        remoteip: req.ip
      })
    );

    if (!response.data.success) {
      return res.status(403).json({
        success: false,
        message: "Captcha verification failed"
      });
    }
  }
}
```



Reply to ChatGPT



Share a link to chat?

This creates a copy that others can chat with



```

    }

    const response = await axios.post(
      "https://challenges.cloudflare.com/turnstile/v0/siteverify",
      new URLSearchParams({
        secret:
process.env.TURNSTILE_SECRET_KEY,
        response: token,
        remoteip: req.ip
      })
    );

    if (!response.data.success) {
      return res.status(403).json({
        success: false,
        message: "Captcha verification failed"
      });
    }

    next();
  } catch (error) {
    return res.status(500).json({
      success: false,
      message: "Captcha verification error"
    });
  }
}

module.exports = verifyCaptcha;

```

Use CAPTCHA mainly for public endpoints:

Demo request

Contact form

Newsletter

Login after repeated fail



Forgot password

Study

AI + Reply to ChatGPT





Thinking

verifyCaptcha?



Use CAPTCHA mainly for public endpoints:

Demo request

Contact form

Newsletter

Login after repeated failures

Forgot password

Student/teacher registration

AI report request if public/exposed

Do not use CAPTCHA as a replacement for authentication and authorization.

4. Can Direct API Calls from Browser or Terminal CURL Be Prevented?

You cannot fully prevent direct API calls. Any public API can be called using curl, Postman, scripts, or browser fetch.

What you can do is make direct API calls useless unless they pass strict backend checks.

You cannot rely on:

Frontend route guards

Hidden buttons

Dis



Reply to ChatGPT



CA

Checking origin only



Thinking



You cannot rely on:

Frontend route guards
Hidden buttons
Disabled form fields
CAPTCHA only on frontend
Checking Origin only
Checking Referer only
Obfuscating API URLs

You must enforce:

Authentication
Authorization
Role-based access
School-based access
CSRF protection for cookie sessions
Rate limiting
Input validation
CAPTCHA server verification for public APIs
API key or service token for internal APIs
Signed URLs for file access
Audit logs
IP/device anomaly detection

Example:

JavaScript



```
router.post(  
  "/students",  
  authenticate
```



Reply to ChatGPT





Thinking



Example:

JavaScript



```
router.post(
  "/students",
  authenticate,
  authorize(["SCHOOL_ADMIN", "PRINCIPAL"]),
  validateCreateStudent,
  createStudent
);
```

Even if someone calls:

Bash



```
curl -X POST
https://api.sproutmentors.com/students
```

It should fail unless they have:

Valid token

Correct role

Correct schoolId

Valid payload

Valid consent fields

No forbidden fields

5. Server-Side Validation to Prevent Attack



Reply to ChatGPT





Thinking



5. Server-Side Validation to Prevent Attack

Yes, server-side validation is mandatory.

Frontend validation is only for user experience.
Backend validation is for security.

A. Validate every request body

Use zod, joi, yup, or express-validator.

Example using Zod:

JavaScript



```
const { z } = require("zod");

const createStudentSchema = z.object({
  name: z.string().trim().min(2).max(100),
  rollNo: z.string().trim().max(20),
  admissionNo: z.string().trim().max(50),
  class: z.string().trim().max(20),
  section: z.string().trim().max(10),
  gender: z.enum(["Male", "Female", "Other"]),
  dob: z.coerce.date(),
  dateOfJoin: z.coerce.date(),

  parents: z.object({
    fatherName:
      z.string().trim().max(100).optional(),
    fatherMobile: z.string().regex(/^\+?[1-9]\d{7,14}$/).optional(),
    fatherEmail:
      z.string().email().optional(),
    motherName:
      z.string().trim().max(100).optional(),
```



Reply to ChatGPT





Thinking



Example using Zod:

JavaScript



```
const { z } = require("zod");

const createStudentSchema = z.object({
  name: z.string().trim().min(2).max(100),
  rollNo: z.string().trim().max(20),
  admissionNo: z.string().trim().max(50),
  class: z.string().trim().max(20),
  section: z.string().trim().max(10),
  gender: z.enum(["Male", "Female", "Other"]),
  dob: z.coerce.date(),
  dateOfJoin: z.coerce.date(),

  parents: z.object({
    fatherName:
z.string().trim().max(100).optional(),
    fatherMobile: z.string().regex(/^\\+?[1-9]\\d{7,14}$/).optional(),
    fatherEmail:
z.string().email().optional(),
    motherName:
z.string().trim().max(100).optional(),
    motherMobile: z.string().regex(/^\\+?[1-9]\\d{7,14}$/).optional(),
    motherEmail: z.string().email().optional()
  }),

  parentConsent: z.object({
    accepted: z.boolean().refine(v => v === true),
    acceptedBy: z.string().min(2).max(100),
    relation: z.enum(["FATHER", "MOTHER", "GUARDIAN"])
  })
});
```

Middleware:



Reply to ChatGPT





Thinking



```

    acceptedBy: z.string().min(2).max(100),
    relation: z.enum(["FATHER", "MOTHER",
"GUARDIAN"])
  })
});

```

Middleware:

JavaScript



```

function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);

    if (!result.success) {
      return res.status(400).json({
        success: false,
        message: "Validation failed",
        errors: result.error.flatten()
      });
    }

    req.validatedBody = result.data;
    next();
  };
}

```

B. Prevent NoSQL injection

Bad:

JavaScript



```

const user = await User.findOne({
  email: req.body.email,

```



Reply to ChatGPT





Thinking



B. Prevent NoSQL injection

Bad:

JavaScript



```
const user = await User.findOne({  
  email: req.body.email,  
  password: req.body.password  
});
```

Attack:

JSON



```
{  
  "email": { "$ne": null },  
  "password": { "$ne": null }  
}
```

Protection:

JavaScript



```
const mongoSanitize =  
  require("express-mongo-sanitize");  
  
app.use(mongoSanitize());
```

Also type-check email as string:



Reply to ChatGPT



```
if (typeof req.body.email !== "string") {  
  // ...  
}
```



Thinking



Also type-check email as string:

JavaScript



```
if (typeof req.body.email !== "string") {  
  return res.status(400).json({ message:  
    "Invalid email" });  
}
```

C. Prevent mass assignment

Do not do this:

JavaScript



```
await Student.create(req.body);
```

Because attacker can send:

JSON



```
{  
  "name": "Test",  
  "role": "SUPER_ADMIN",  
  "schoolId": "another_school_id",  
  "isDeleted": false  
}
```

Do this:



Reply to ChatGPT





Thinking



Do this:

JavaScript



```
const payload = {
  name: req.validatedBody.name,
  rollNo: req.validatedBody.rollNo,
  admissionNo: req.validatedBody.admissionNo,
  class: req.validatedBody.class,
  section: req.validatedBody.section,
  schoolId: req.user.schoolId,
  createdBy: req.user._id
};

await Student.create(payload);
```

D. Prevent cross-school access

Bad:

JavaScript



```
const student = await
Student.findById(req.params.id);
```

Good:

JavaScript



```
const student = await Student.findOne({
  _id: req.params.id,
  schoolId: req.user.schoolId,
```



Reply to ChatGPT





Thinking



Good:

JavaScript



```
const student = await Student.findOne({
  _id: req.params.id,
  schoolId: req.user.schoolId,
  isDeleted: false
});
```

This is one of the most important checks for SproutMentors.

E. Validate ObjectId

JavaScript



```
const mongoose = require("mongoose");

function validateObjectId(paramName) {
  return (req, res, next) => {
    if (!
mongoose.Types.ObjectId.isValid(req.params[par
amName])) {
      return res.status(400).json({
        success: false,
        message: "Invalid ID"
      });
    }
    next();
  };
}
```



Reply to ChatGPT



F. File upload validation



Thinking



F. File upload validation

For profile photos, marksheets, documents, certificates:

JavaScript



```
const allowedMimeTypes = [
  "image/jpeg",
  "image/png",
  "application/pdf"
];

if (!allowedMimeTypes.includes(file.mimetype)) {
  throw new Error("Invalid file type");
}

if (file.size > 5 * 1024 * 1024) {
  throw new Error("File too large");
}
```

Also scan files using antivirus or a malware scanning service before making them available.

6. Middleware Implementation and Security Check in Node.js

You should implement middleware in layers.

Recommended middleware order



Reply to ChatGPT



```
const express = require('express');
```




Thinking



Recommended middleware order

JavaScript



```
const express = require("express");
const helmet = require("helmet");
const cors = require("cors");
const rateLimit =
  require("express-rate-limit");
const mongoSanitize =
  require("express-mongo-sanitize");
const hpp = require("hpp");
const cookieParser = require("cookie-parser");

const app = express();

app.set("trust proxy", 1);

app.use(helmet());

app.use(cors({
  origin: [
    "https://app.sproutmentors.com",
    "https://sproutmentors.com",
    "https://www.sproutsong.com"
  ],
  credentials: true,
  methods: ["GET", "POST", "PUT", "PATCH",
    "DELETE"],
  allowedHeaders: ["Content-Type",
    "Authorization", "X-CSRF-Token"]
}));

app.use(express.json({ limit: "1mb" }));
app.use(express.urlencoded({ extended: true,
  limit: "1mb" }));
app.use(cookieParser());

app.use(mongoSanitize());
app.use(hpp());
```



Reply to ChatGPT





Thinking



Rate limiting

JavaScript



```
const generalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 300,
  standardHeaders: true,
  legacyHeaders: false
});

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: "Too many login attempts. Please
try again later."
});

app.use("/api", generalLimiter);
app.use("/api/auth/login", authLimiter);
app.use("/api/auth/forgot-password",
authLimiter);
```

Authentication middleware

JavaScript



```
const jwt = require("jsonwebtoken");

function authenticate(req, res, next) {
  try {
    const header = req.headers.authorization;

    if (!header || !header.startsWith("Bearer
")) {
      return res.status(401).json({
```



Reply to ChatGPT





Thinking



Authentication middleware

JavaScript



```
const jwt = require("jsonwebtoken");

function authenticate(req, res, next) {
  try {
    const header = req.headers.authorization;

    if (!header || !header.startsWith("Bearer ")) {
      return res.status(401).json({
        success: false,
        message: "Authentication required"
      });
    }

    const token = header.split(" ")[1];

    const decoded = jwt.verify(token,
process.env.JWT_SECRET, {
  algorithms: ["HS256"]
});

    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: "Invalid or expired token"
    });
  }
}
```

Reply to ChatGPT



Reply to ChatGPT



JavaScript



Thinking



Role authorization middleware

JavaScript



```
function authorize(allowedRoles = []) {
  return (req, res, next) => {
    if (!req.user || !
allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: "Access denied"
      });
    }

    next();
  };
}
```

Permission middleware

For your modules, role alone is not enough. You need permission-level checks.

JavaScript



```
function requirePermission(permission) {
  return async (req, res, next) => {
    const permissions = req.user.permissions
    || [];

    if (!permissions.includes(permission)) {
      return res.status(403).json({
        success: false,
        message: "Permission denied"
      });
    }

    next();
  };
}
```



Reply to ChatGPT





Thinking



Permission middleware

For your modules, role alone is not enough. You need permission-level checks.

JavaScript



```
function requirePermission(permission) {  
  return async (req, res, next) => {  
    const permissions = req.user.permissions  
    || [];  
  
    if (!permissions.includes(permission)) {  
      return res.status(403).json({  
        success: false,  
        message: "Permission denied"  
      });  
    }  
  
    next();  
  };  
}
```

Example permissions:

Plain text



```
students.read  
students.create  
students.update  
students.delete  
attendance.mark  
attendance.update  
reports.download  
teacher.bank.view  
teacher.medical.view  
ai.report.generate
```



Reply to ChatGPT



School isolation middleware



Thinking



School isolation middleware

JavaScript



```
function enforceSchoolScope(model, idParam = "id") {
  return async (req, res, next) => {
    const record = await model.findOne({
      _id: req.params[idParam],
      schoolId: req.user.schoolId,
      isDeleted: { $ne: true }
    });

    if (!record) {
      return res.status(404).json({
        success: false,
        message: "Record not found"
      });
    }

    req.scopedRecord = record;
    next();
  };
}
```

Audit log middleware

JavaScript



```
function auditLog(action, moduleName) {
  return async (req, res, next) => {
    const oldJson = res.json;

    res.json = async function (data) {
      try {

```



Reply to ChatGPT





Thinking



Audit log middleware

JavaScript



```
function auditLog(action, moduleName) {
  return async (req, res, next) => {
    const oldJson = res.json;

    res.json = async function (data) {
      try {
        await AuditLog.create({
          actorId: req.user?._id,
          actorRole: req.user?.role,
          schoolId: req.user?.schoolId,
          module: moduleName,
          action,
          method: req.method,
          path: req.originalUrl,
          recordId: req.params.id,
          ipAddress: req.ip,
          userAgent:
req.headers["user-agent"],
          statusCode: res.statusCode,
          createdAt: new Date()
        });
      } catch (e) {}

      return oldJson.call(this, data);
    };

    next();
  };
}
```

Use this on sensitive APIs:



Reply to ChatGPT





Thinking



Use this on sensitive APIs:

JavaScript



```
router.get(
  "/students/:id",
  authenticate,
  authorize(["SCHOOL_ADMIN", "PRINCIPAL",
    "TEACHER"]),
  validateObjectId("id"),
  auditLog("VIEW_STUDENT", "students"),
  getStudent
);
```

7. MongoDB Data Protection

Your database is MongoDB 6.0.27 based on the uploaded prelude metadata. `prelude.json`

A. Enable authentication

MongoDB must not run without authentication.

Use a dedicated app user, not root.

JavaScript



```
db.createUser({
  user: "sprout_app_user",
  pwd: "STRONG_PASSWORD",
```



Reply to ChatGPT





Thinking

user, not root.



JavaScript



```
db.createUser({
  user: "sprout_app_user",
  pwd: "STRONG_PASSWORD",
  roles: [
    { role: "readWrite", db:
"sproutmentors_prod" }
  ]
});
```

Do not use root credentials in the Node.js app.

B. Bind MongoDB to private network only

MongoDB should not be exposed publicly.

In Docker Compose:

YAML



```
mongodb:
  image: mongo:6
  ports:
    - "127.0.0.1:27017:27017"
```

Or better, expose Mongo only inside Docker network and not on host.



Reply to ChatGPT





Thinking



C. Use TLS if Mongo is remote

If app and database are on different servers, use TLS connection.

Connection string:

Plain text



```
mongodb://user:pass@host:27017/db?authSource=admin&tls=true
```

D. Encrypt sensitive fields before saving

Example with Node.js crypto:

JavaScript



```
const crypto = require("crypto");

const algorithm = "aes-256-gcm";
const key =
  Buffer.from(process.env.FIELD_ENCRYPTION_KEY,
    "hex");

function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const cipher =
    crypto.createCipheriv(algorithm, key, iv);

  const encrypted = Buffer.concat([
```



Reply to ChatGPT





Thinking



```
const crypto = require("crypto");

const algorithm = "aes-256-gcm";
const key =
Buffer.from(process.env.FIELD_ENCRYPTION_KEY,
"hex");

function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const cipher =
crypto.createCipheriv(algorithm, key, iv);

  const encrypted = Buffer.concat([
    cipher.update(String(text), "utf8"),
    cipher.final()
  ]);

  const tag = cipher.getAuthTag();

  return {
    iv: iv.toString("hex"),
    content: encrypted.toString("hex"),
    tag: tag.toString("hex")
  };
}

function decrypt(payload) {
  const decipher = crypto.createDecipheriv(
    algorithm,
    key,
    Buffer.from(payload.iv, "hex")
  );

  decipher.setAuthTag(Buffer.from(payload.tag,
"hex"));

  const decrypted = Buffer.concat([
    decipher.update(Buffer.from(payload.content,
"hex")),
    decipher.final()
  ]);

  return decrypted.toString("utf8");
}
```

Use



Reply to ChatGPT





Thinking



```
const decrypted = Buffer.concat([
  decipher.final()
]);

return decrypted.toString("utf8");
}
```

Use encryption for:

Teacher bank account

Aadhaar

PAN

Passport

Driving license

Medical notes

Student parent phone/email if required

Counselling notes

Sensitive AI reports

E. Use field projection

Never return full records blindly.

Bad:

JavaScript



```
const teacher = await
TeacherProfile.findById(id);
res.json(teacher);
```

Good:



Reply to ChatGPT



JavaScript





Thinking



E. Use field projection

Never return full records blindly.

Bad:

JavaScript



```
const teacher = await
TeacherProfile.findById(id);
res.json(teacher);
```

Good:

JavaScript



```
const teacher = await
TeacherProfile.findById(id)
.select("teacherInfo
personalInfo.profilePhoto academicRecords
teachingExperience status");
```

For sensitive fields:

JavaScript



```
const bankDetails = await
TeacherBankDetails.findOne({
teacherId,
schoolId: req.user.schoolId
}).select("+accountNumberEnc +ifsc");
```



Only allow this route to HR/payroll roles.



Reply to ChatGPT





Thinking



F. Prevent accidental sensitive data exposure in Mongoose

Use `select: false`.

JavaScript



```
const teacherBankSchema = new
mongoose.Schema({
  teacherId: { type:
mongoose.Schema.Types.ObjectId, ref:
"Teacher", index: true },
  schoolId: { type:
mongoose.Schema.Types.ObjectId, index: true },

  accountNumberEnc: { type: Object, select:
false },
  accountNumberLast4: String,
  ifscEnc: { type: Object, select: false },
  bankName: String
});
```

G. Add compound indexes with `schoolId`

Current metadata shows some unique indexes are global, such as `students.admissionNo` and `classes.classId`. `students.metadata.json` `classes.metadata.json`

For SaaS/multi-school setup, avoid global uniqueness.

Better:



Reply to ChatGPT





Thinking



Better:

JavaScript



```
db.students.createIndex(  
  { schoolId: 1, admissionNo: 1 },  
  { unique: true }  
);  
  
db.classes.createIndex(  
  { schoolId: 1, classId: 1, academicYearId:  
  1 },  
  { unique: true }  
);
```

Why: two different schools may both have ADM001 or 10-A.

H. Backups must be encrypted

Backup command:

Bash



```
mongodump \  
  --uri="$MONGO_URI" \  
  --archive="/backups/sproutmentors-$(date +  
  %F).archive" \  
  --gzip
```

Then encrypt:



Reply to ChatGPT





Thinking



Bash



```
gpg -c /backups/  
sproutmentors-2026-05-20.archive
```

Also:

Keep backup access limited.

Rotate backups.

Test restore monthly.

Do not use production backup in development without masking.

I. Data masking for development

Never use real student/teacher production data in development.

Mask script logic:

JavaScript



```
db.students.find().forEach(doc => {  
  db.studeOne(  
    { _id: doc._id },  
    {  
      $set: {  
        name: "Student " +  
doc._id.toString().s ↓ 4),  
        "parents.fath oile": "9999999999",  
        "parents.fatherEmail":  
"masked@example.com"      }  
    }  
  )  
})
```



Reply to ChatGPT





Thinking



JavaScript



```
db.students.find().forEach(doc => {
  db.students.updateOne(
    { _id: doc._id },
    {
      $set: {
        name: "Student " +
doc._id.toString().slice(-4),
        "parents.fatherMobile": "9999999999",
        "parents.fatherEmail":
"masked@example.com",
        "parents.motherMobile": "9999999998",
        "parents.motherEmail":
"masked@example.com"
      }
    }
  );
});
```

Collection-wise Technical Recommendations

Students

Current risk: high.

Technical fixes:

Remove loginDetails.password.

Move login to users collection.

Add schoolId, academicYearId, consentRefId.

Convert dob and dateOfJoin to Date.



Reply to ChatGPT



Add compound index (schoolId, admissionNo).



Thinking



Remove `loginDetails.password`.
Move login to `users` collection.
Add `schoolId`, `academicYearId`, `consentRefId`.
Convert `dob` and `dateOfJoin` to `Date`.
Use parent phone/email validation.
Encrypt parent contact if highly sensitive.
Add compound index { `schoolId`, `admissionNo` }.
Do not return parent data unless role requires it.

Teachers

Current risk: critical because of `plainPassword`.

Technical fixes:

Remove `plainPassword`.
Rename `password` to `passwordHash`.
Use `select: false`.
Add login security fields.
Add MFA for admin/teacher accounts.
Add failed login tracking.

Teacher Profiles

Current risk: very critical.

Technical fixes:



Reply to ChatGPT



Spe
collections.



Thinking



Teacher Profiles

Current risk: very critical.

Technical fixes:

Split bank, medical, ID, family data into separate collections.

Encrypt Aadhaar, PAN, passport, driving license, account number.

Use `select: false`.

Restrict access by permission.

Add audit logs for every sensitive view.

Avoid collecting caste/religion unless legally required.

Mask all sensitive fields in API response.

Students Attendance

Current risk: high.

Technical fixes:

Add `schoolId`, `academicYearId`.

Replace `free-text` `reason` with `reasonCategory`.

Keep `reasonNote` restricted.

Add `markedBy`, `updatedBy`, `editHistory`.

Add compound index { `schoolId`, `class`, `section`, `date` }.

Prevent attendance update after lock date unless

adm



Reply to ChatGPT





Thinking



Prevent attendance update after lock date unless admin approves.

Classes

Current risk: medium.

Technical fixes:

Add `schoolId`, `academicYearId`.

Replace embedded monitor details with `monitorStudentId`.

Use `ObjectId` for `teacherId`.

Do not expose monitor email/image unless needed.

Use structured schedule with `subjectId`, `teacherId`, start/end time.

Substitutions

Current risk: medium.

Technical fixes:

Use `originalTeacherId` and `substituteTeacherId`.

Avoid plain teacher name as primary reference.

Replace reason free text with enum.

Add approval workflow.

Ad



Reply to ChatGPT





Thinking



Replace reason free text with enum.

Add approval workflow.

Add indexes for date/class/teacher.

Syllabus Progress

Current risk: medium.

Technical fixes:

Add `schoolId`, `academicYearId`, `termId`.

Use `subjectId` instead of plain subject string.

Add `updatedBy`, `verifiedBy`.

Do not use syllabus progress directly as teacher performance without notice.

Contacts

Current risk: medium.

Technical fixes:

Add consent fields.

Add source and purpose.

Add communication opt-in.

Add retention/delete fields.

Add rate limiting and CAPTCHA server verification.

Add indexes for email, mobile, createdAt



Reply to ChatGPT





Thinking



Add retention/delete fields.

Add rate limiting and CAPTCHA server verification.

Add indexes for email, mobile, createdAt.

Mentors / Educators

Current risk: medium to high.

Technical fixes:

Keep unique email index, but add `schoolId` if they are school-specific.

Add consent fields.

Add role and access scope.

Add profile visibility setting.

Do not expose email/mobile publicly by default.

AI Trainings / Insights

Current risk depends on whether student data is used.

Technical fixes:

Add AI processing logs.

Store anonymized references where possible.

Do not send full student PII to AI providers.

Add human review field.

Add consent reference.



Reply to ChatGPT





Thinking logs.



Store anonymized references where possible.

Do not send full student PII to AI providers.

Add human review field.

Add consent reference.

Add prompt/output retention policy.

Direct Answers to Your 7 Questions

1. Are relevant fields missing?

Yes. Main missing fields are:

`schoolId, academicYearId, createdBy, updatedBy, deletedAt, isDeleted, consentRefId, privacyPolicyVersion, retentionUntil, audit fields, approval fields, and role/permission fields.`

For DPDP, consent and retention fields are especially important.

2. Are types correct?

Some are okay, but several should change:

String IDs → ObjectId

Date

Free

Free-text reason → reasonCategory enum + restricted

note



Reply to ChatGPT



Yes. Main missing fields are:



Thinking



SymbolicYearId, createdBy, updatedAt, deletedAt, isDeleted, consentRefId, privacyPolicyVersion, retentionUntil, audit fields, approval fields, and role/permission fields.

For DPDP, consent and retention fields are especially important.

2. Are types correct?

Some are okay, but several should change:

String IDs → ObjectId

Date strings → Date

Free-text status → enum

Free-text reason → reasonCategory enum + restricted note

Password/plainPassword → passwordHash only

Sensitive ID/bank/medical fields → encrypted object/string with masked display field

Percentage/CGPA → Number with min/max validation

Mobile → String with regex

Email → lowercase string with email validation

3. Does CAPTCHA provide frontend security?



Reply to ChatGPT



No. Frontend CAPTCHA only slows bots using the server. Frontend CAPTCHA only slows bots using the UI. It does not stop curl/postman/direct API calls.



Thinking



3. Does CAPTCHA provide frontend security?

No, not by itself. CAPTCHA must be verified on the server. Frontend CAPTCHA only slows bots using the UI. It does not stop curl/Postman/direct API calls.

4. Can direct API calls be prevented?

Not fully. Any public API can be called directly. But you can block unauthorized direct API calls through backend authentication, authorization, role checks, school scoping, rate limiting, server-side CAPTCHA verification, CSRF protection, and validation.

5. Is server-side validation required?

Yes. Every endpoint must validate body, params, query, files, role, schoolId, ownership, and permission. Never trust frontend validation.

6. Is middleware needed in Node.js?



Yes. You need middleware for:



Reply to ChatGPT



Share a link to chat?

This creates a copy that others can chat with



6. Is middleware needed in Node.js?

Yes. You need middleware for:

- Authentication
- Authorization
- Permission checks
- School isolation
- Validation
- Rate limiting
- CAPTCHA verification
- Mongo sanitization
- File validation
- Audit logging
- Error handling
- CSRF if using cookies
- Helmet/security headers

7. How should MongoDB data be protected?

Use:

- MongoDB authentication
- Private network binding

Le
En



Reply to ChatGPT



Field-level encryption for sensitive data



Thinking



7. How should MongoDB data be protected?

Use:

- MongoDB authentication
- Private network binding
- Least-privilege DB user
- Encrypted backups
- Field-level encryption for sensitive data
- Projection and `select: false`
- Compound indexes with `schoolId`
- Soft delete and retention policies
- Audit logs
- No production data in development
- No root DB credentials in app
- No public MongoDB port

Final Technical Priority List

1. Remove `plainPassword` and `loginDetails.password` immediately.
2. Add `schoolId` to all operational collections.
3. Add backend authorization with school-level scoping.
4. Split `teacherprofiles` into separate sensitive collections.

5



Reply to ChatGPT



6

Add consent and privacy version fields

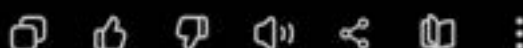


Thinking



Final Technical Priority List

1. Remove `plainPassword` and `loginDetails.password` immediately.
2. Add `schoolId` to all operational collections.
3. Add backend authorization with school-level scoping.
4. Split `teacherprofiles` into separate sensitive collections.
5. Encrypt teacher bank, identity, medical, and family data.
6. Add consent and privacy version fields.
7. Replace free-text sensitive fields with enums and restricted notes.
8. Add audit logs for view/edit/delete/download/export.
9. Add server-side validation for every route.
10. Add middleware stack: Helmet, CORS, rate limit, mongo sanitize, auth, permission, audit.
11. Use private MongoDB networking and encrypted backups.
12. Do not expose full documents in API responses.



Reply to ChatGPT

